

Coupling and Cohesion

Coupling

Coupling refers to the degree of interdependence between software modules. It measures how closely connected different modules are within a system. There are different types of coupling, generally classified by their tightness. In general, lower coupling (loose coupling) is preferred because it makes the system more modular and easier to maintain.

Types of Coupling:

1. Content Coupling (Highest)

- One module directly modifies or relies on the internal workings of another module (e.g., accessing or modifying its data).
- **Example:** Module A changes the local data of Module B.

2. Common Coupling

- Multiple modules share the same global data.
- **Example:** Two or more modules access and modify a global variable.

3. External Coupling

- Modules depend on external factors (e.g., hardware, software libraries, protocols).
- **Example:** Two modules communicating through an external interface or hardware device.

4. Control Coupling

- One module controls the behavior of another by passing control information.
- **Example:** A module passes a flag to another module to dictate its behavior.

5. Stamp Coupling (Data-Structured Coupling)

- Modules share a composite data structure, and only part of the data is used by the receiving module.
- **Example:** Passing an entire record to a function that only needs one field of the record.

6. Data Coupling (Lowest)

- Modules share data through parameters. Each parameter is an elementary piece of data.
- **Example:** Passing individual variables to a function.

Importance of Low Coupling:

- **Maintainability:** Easier to change one module without affecting others.
- **Reusability:** Modules can be reused in different systems.
- **Testability:** Modules can be tested independently.
- **Scalability:** System can be scaled more easily.

Cohesion

Cohesion refers to the degree to which the elements inside a module belong together. It measures how closely related the responsibilities of a single module are. Higher cohesion within a module is preferred because it makes the module more understandable and maintainable.

Types of Cohesion:

1. Coincidental Cohesion (Lowest)

- Elements are grouped arbitrarily; they have no meaningful relationship.
- **Example:** A module that performs unrelated tasks.

2. Logical Cohesion

- Elements are grouped because they perform similar activities or are related logically.
- **Example:** A module that contains all input/output operations, regardless of their specific purposes.

3. Temporal Cohesion

- Elements are grouped by when they are processed (e.g., initialization, cleanup).
- **Example:** A module that initializes different unrelated parts of the system.

4. Procedural Cohesion

- Elements are grouped because they follow a specific sequence of execution.
- **Example:** A module that processes a sequence of steps in a specific order.

5. Communicational Cohesion

- Elements are grouped because they operate on the same data or contribute towards a single task.
- **Example:** A module that processes and outputs the same data.

6. Sequential Cohesion

- Elements are grouped because the output from one part is the input to another part.
- **Example:** A module that reads data from a file, processes it, and writes it to another file.

7. Functional Cohesion (Highest)

- Elements are grouped because they all contribute to a single, well-defined task.
- **Example:** A module that performs a specific calculation.

Importance of High Cohesion:

- **Understandability:** Easier to understand what a module does.
- **Maintainability:** Easier to make changes without affecting unrelated parts.
- **Reusability:** Modules with a single responsibility can be reused in different contexts.
- **Reliability:** Modules are more robust and less prone to errors.

Summary

- **Coupling:** Measures the degree of interdependence between modules. Lower coupling is desirable for better modularity and maintainability.
- **Cohesion:** Measures how closely related the functions within a single module are. Higher cohesion is desirable for clarity, maintainability, and reliability.

By aiming for low coupling and high cohesion, software engineers can create systems that are easier to understand, maintain, and extend.